

你指定的这版 `basic_mpc`，本质上是一个：

单水箱、连续流量、带约束、滚动优化的基础 MPC。

它现在解决的不是电价问题，也不直接决定具体哪台水泵开关，而是在每个小时回答：

“未来 24 小时，泵站每小时应该提供多大比例的总流量，才能让水位安全、接近目标，而且不要突然大幅改变？”

1. 先建立直觉：MPC 在做什么

可以把 MPC 想成一个不断重做计划的水库管理员。

当前是第 k 个小时，它会：

1. 测量当前水位 $x(k)$ ；
2. 获取未来 24 小时的预测用水量；
3. 假设不同的供水方案；
4. 计算每套方案下未来水位；
5. 找出代价最小、又不违反约束的方案；
6. 只执行第一个小时；
7. 一小时后重新测量，再规划一次未来 24 小时。

重点是：

MPC 虽然计划 24 小时，但每次只执行第一步。

这叫作滚动时域控制 (Receding Horizon Control)。

比如当前在 00:00，MPC 算出：

$$\mathbf{U}^* = \begin{bmatrix} 0.35 \\ 0.50 \\ 0.55 \\ \vdots \end{bmatrix}$$

这代表未来 24 小时建议的归一化流量。但现在只执行：

$$u(0) = 0.35$$

到 01:00 后，根据新水位重新计算一套计划。昨天算出的第二步 0.50 不会被机械照搬。

2. 这套模型里的变量

水位

$$x(k)$$

表示第 k 个小时开始时的水箱水位，单位是米。

当前参数：

$$1.40 \leq x(k) \leq 3.37$$

初始水位：

$$x(0) = 2.30 \text{ m}$$

MPC 控制量

$$u(k)$$

是归一化的期望总流量：

$$0 \leq u(k) \leq 1$$

它不是某台泵的开关状态。

- $u = 0$: 要求零流量;
- $u = 0.5$: 要求最大流量的 50%;
- $u = 1$: 要求最大流量。

最大流量是:

$$q_{\max} = 0.05 \text{ m}^3/\text{s}$$

所以 MPC 要求的实际流量为:

$$q_{\text{request}}(k) = q_{\max} u(k)$$

例如:

$$u(k) = 0.6$$

对应:

$$q_{\text{request}}(k) = 0.05 \times 0.6 = 0.03 \text{ m}^3/\text{s}$$

用水需求

$$d(k)$$

表示用户从水箱中取走的流量, 单位也是:

$$\text{m}^3/\text{s}$$

注意, 这里的 demand 是用水需求, 不是电力系统的 TOTALDEMAND。

3. 最基础的数学原理: 质量守恒

水箱模型来自非常简单的守恒关系:

$$\text{储水量变化} = \text{流入水量} - \text{流出水量}$$

假设水箱横截面积为 A , 水位为 x , 那么水箱体积是:

$$V = Ax$$

一个采样周期内:

$$V(k+1) - V(k) = \Delta t (q_{\text{in}}(k) - d(k))$$

因为 $V = Ax$, 所以:

$$A[x(k+1) - x(k)] = \Delta t (q_{\text{in}}(k) - d(k))$$

整理得到:

$$x(k+1) = x(k) + \frac{\Delta t}{A} (q_{\text{in}}(k) - d(k))$$

而基础 MPC 假设:

$$q_{\text{in}}(k) = q_{\max} u(k)$$

因此预测模型是:

$$x(k+1) = x(k) + \frac{\Delta t}{A} (q_{\max} u(k) - d(k))$$

这就是整个基础 MPC 最核心的系统模型。

4. 把当前参数代进去

代码中的参数为：

$$\Delta t = 3600 \text{ s}$$

$$A = 1000 \text{ m}^2$$

$$q_{\max} = 0.05 \text{ m}^3/\text{s}$$

所以：

$$\frac{\Delta t}{A} = \frac{3600}{1000} = 3.6$$

模型变成：

$$x(k+1) = x(k) + 3.6(0.05u(k) - d(k))$$

进一步写成：

$$x(k+1) = x(k) + 0.18u(k) - 3.6d(k)$$

这两个系数很好理解。

如果水泵满流量运行，且没有用水：

$$u = 1, \quad d = 0$$

那么一小时后水位升高：

$$\Delta x = 0.18 \text{ m}$$

如果用水需求为：

$$d = 0.025 \text{ m}^3/\text{s}$$

为了让水位基本不变，需要：

$$q_{\max}u = d$$

所以：

$$u = \frac{d}{q_{\max}} = \frac{0.025}{0.05} = 0.5$$

这意味着：

当需求为 $0.025 \text{ m}^3/\text{s}$ 时，水泵提供 50% 的最大流量，水位理论上保持不变。

如果只给 $u = 0.4$ ，则：

$$\begin{aligned} \Delta x &= 3.6(0.05 \times 0.4 - 0.025) \\ &= 3.6(0.020 - 0.025) = -0.018 \text{ m} \end{aligned}$$

一小时后水位下降 1.8 厘米。

5. MPC 为什么要预测 24 步

普通反馈控制可能只看当前误差：

$$e(k) = x_{\text{ref}}(k) - x(k)$$

水位低了就加大供水，水位高了就减少供水。

MPC 更进一步：它会考虑未来需求。

例如当前水位看起来正常，但 MPC 知道未来三小时需求会突然增大，它可以提前把水位稍微提高。这样高需求到来时，不容易跌破最低水位。

预测第 1 小时：

$$x_{1|k} = x(k) + \frac{\Delta t}{A} (q_{\max} u_{0|k} - \hat{d}_{0|k})$$

预测第 2 小时：

$$x_{2|k} = x_{1|k} + \frac{\Delta t}{A} (q_{\max} u_{1|k} - \hat{d}_{1|k})$$

把第一式代进去：

$$x_{2|k} = x(k) + \frac{\Delta t}{A} [q_{\max}(u_{0|k} + u_{1|k}) - (\hat{d}_{0|k} + \hat{d}_{1|k})]$$

一般地：

$$x_{i|k} = x(k) + \frac{\Delta t}{A} \sum_{j=0}^{i-1} (q_{\max} u_{j|k} - \hat{d}_{j|k})$$

符号 $x_{i|k}$ 的意思是：

在当前时刻 k ，对未来第 i 步水位作出的预测。

控制器一次求解 24 个变量：

$$U = \begin{bmatrix} u_{0|k} \\ u_{1|k} \\ \vdots \\ u_{23|k} \end{bmatrix}$$

6. MPC 在优化什么

这套基础 MPC 的目标函数可以写成：

$$J = \sum_{i=1}^N w_x (x_{i|k} - r_{i|k})^2 + \sum_{i=0}^{N-1} w_u u_{i|k}^2 + \sum_{i=0}^{N-1} w_{\Delta u} (u_{i|k} - u_{i-1|k})^2$$

其中：

- $N = 24$ ：预测 24 小时；
- $r_{i|k}$ ：目标水位；
- $w_x = 50$ ：水位跟踪权重；
- $w_u = 0.02$ ：输入大小权重；
- $w_{\Delta u} = 0.50$ ：输入变化权重。

可以把它理解成三张罚单。

第一张罚单：水位偏离目标

$$J_x = w_x \sum_{i=1}^N (x_{i|k} - r_{i|k})^2$$

水位离参考值越远，罚得越多。

之所以平方，是因为：

- 正误差和负误差都要惩罚；
- 大误差会受到更强的惩罚；
- 数学上仍然容易优化。

例如误差从 0.1 m 增大到 0.2 m，平方项从：

$$0.1^2 = 0.01$$

变成：

$$0.2^2 = 0.04$$

误差扩大 2 倍，惩罚扩大 4 倍。

第二张罚单：供水指令太大

$$J_u = w_u \sum_{i=0}^{N-1} u_{i|k}^2$$

它会阻止控制器无意义地一直要求大流量。

但要说清楚：

这一项不是真正的电费。

它没有使用电价，也没有使用泵功率曲线。这里只是简单地倾向于较小的控制输入。

第三张罚单：流量变化太剧烈

定义：

$$\Delta u_{0|k} = u_{0|k} - u(k-1)$$

$$\Delta u_{i|k} = u_{i|k} - u_{i-1|k}$$

于是：

$$J_{\Delta u} = w_{\Delta u} \sum_{i=0}^{N-1} (\Delta u_{i|k})^2$$

它的作用是让控制信号平滑。

没有这一项时，控制器可能出现：

$$0, 1, 0, 1, 0, 1$$

有了变化惩罚后，它更倾向于：

$$0.3, 0.45, 0.52, 0.48, 0.40$$

需要注意：不能简单说“50 比 0.5 大 100 倍，所以水位一定重要 100 倍”，因为三个惩罚项的单位和数值尺度不同。权重必须结合实际输出调试。

7. 硬约束与软目标的区别

这是 MPC 里非常重要的一点。

软目标

例如：

$$x(k) \approx r(k)$$

控制器希望水位接近参考值，但不要求完全相等。偏离参考值只会增加目标函数。

硬约束

代码要求：

$$1.40 \leq x_{i|k} \leq 3.37$$

$$0 \leq u_{i|k} \leq 1$$

$$|u_{i|k} - u_{i-1|k}| \leq 0.35$$

这些不是“尽量满足”，而是优化器必须满足。

因此，参考水位为 2.30 m 时，水位可以暂时是 2.20 m；只是会受到惩罚。

但预测水位低于 1.40 m，则该方案直接不允许。

8. 为什么第一小时不能直接从 0 跳到 0.5

初始上一时刻输入是：

$$u(-1) = 0$$

变化约束是：

$$|u(0) - u(-1)| \leq 0.35$$

所以第一小时：

$$u(0) \leq 0.35$$

假设需求恒定为：

$$d = 0.025 \text{ m}^3/\text{s}$$

维持水位原本需要 $u = 0.5$ ，但第一小时只能最多给 0.35。

如果给到上限 $u = 0.35$ ，水位变化为：

$$\Delta x = 3.6(0.05 \times 0.35 - 0.025)$$

$$= 3.6(0.0175 - 0.025) = -0.027 \text{ m}$$

所以第一小时水位会下降约 2.7 厘米。

下一小时控制量可以从 0.35 增加到最多：

$$0.35 + 0.35 = 0.70$$

因此可以逐渐调整到维持水位所需的 $u \approx 0.5$ 。

这正体现了 MPC 的取舍：

它不能只考虑水位，还必须遵守执行变化速度限制。

9. 为什么它能交给 quadprog

状态预测虽然会累积，但仍然是控制变量 U 的线性函数。

定义下三角累加矩阵：

$$L = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 1 & 1 & 0 & \cdots & 0 \\ 1 & 1 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & 1 & \cdots & 1 \end{bmatrix}$$

那么 24 步预测可以整体写成：

$$\mathbf{X} = \mathbf{X}_{\text{base}} + \Phi \mathbf{U}$$

其中：

$$\mathbf{X} = \begin{bmatrix} x_{1|k} \\ x_{2|k} \\ \vdots \\ x_{24|k} \end{bmatrix}$$

$$\mathbf{X}_{\text{base}} = x(k) \mathbf{1} - \frac{\Delta t}{A} L \hat{D}$$

$$\Phi = \frac{\Delta t}{A} q_{\text{max}} L$$

$\mathbf{X}_{\text{base}}$ 可以理解成：

假设未来水泵完全不供水，只考虑用户取水时的水位轨迹。

而：

$$\Phi \mathbf{U}$$

表示水泵供水给水位带来的提升。

输入变化也可以写成矩阵形式：

$$\Delta \mathbf{U} = D \mathbf{U} - \mathbf{p}$$

其中：

$$D = \begin{bmatrix} 1 & 0 & 0 & \cdots \\ -1 & 1 & 0 & \cdots \\ 0 & -1 & 1 & \cdots \\ \vdots & & & \ddots \end{bmatrix}$$

$$\mathbf{p} = \begin{bmatrix} u(k-1) \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

于是目标函数最终能整理成标准二次规划形式：

$$\min_{\mathbf{U}} \quad \frac{1}{2} \mathbf{U}^T \mathbf{H} \mathbf{U} + \mathbf{f}^T \mathbf{U}$$

同时满足：

$$\mathbf{A} \mathbf{U} \leq \mathbf{b}$$

$$\mathbf{U}_{\min} \leq \mathbf{U} \leq \mathbf{U}_{\max}$$

这就是 MATLAB quadprog 所需要的格式。

所以 quadprog 并不理解“水箱”“水泵”这些概念。它只看到：

- 一个二次目标函数；
- 一组线性不等式；
- 一组变量上下界。

真正把水箱问题翻译成这些矩阵的，才是 MPC 控制器。

10. 一轮实际运行逻辑

这套代码默认仿真 72 小时，每次循环代表 1 小时。

在第 k 小时：

第一步：获得未来 24 小时数据

控制器拿到：

$$\hat{d}_{k|k}, \hat{d}_{k+1|k}, \dots, \hat{d}_{k+23|k}$$

以及 24 小时目标水位：

$$r_{k|k}, r_{k+1|k}, \dots, r_{k+23|k}$$

当前版本不是实际预测器，而是生成的周期模拟数据。

第二步：求解 24 小时控制计划

控制器计算：

$$\mathbf{U}^* = \begin{bmatrix} u_{0|k}^* \\ u_{1|k}^* \\ \vdots \\ u_{23|k}^* \end{bmatrix}$$

并同时得到预测水位：

$$\mathbf{X}^* = \begin{bmatrix} x_{1|k}^* \\ x_{2|k}^* \\ \vdots \\ x_{24|k}^* \end{bmatrix}$$

第三步：只发送第一个流量要求

$$u_{\text{request}}(k) = u_{0|k}^*$$

$$q_{\text{request}}(k) = q_{\text{max}} u_{\text{request}}(k)$$

后面 23 步只是当前计划，不立即执行。

第四步：Decision 模块决定实际流量

理论上的真实 Decision 模块应该回答：

- 开哪几台泵；
- 每台泵运行多久；
- 最终实际流量是多少；
- 功率是多少。

但当前只是一个理想模拟模块，直接假设：

$$u_{\text{actual}}(k) = u_{\text{request}}(k)$$

因此：

$$q_{\text{actual}}(k) = q_{\text{request}}(k)$$

第五步：用实际流量更新水位

$$x(k+1) = x(k) + \frac{\Delta t}{A} (q_{\text{actual}}(k) - d(k))$$

这里必须使用实际流量，而不能直接使用 MPC 要求流量。

因为未来真实 Decision 模块可能只能实现某些离散流量，例如 MPC 要求：

$$0.027 \text{ m}^3/\text{s}$$

但实际泵组只能做到：

$$0.025 \text{ m}^3/\text{s}$$

物理水箱只会受到实际 0.025 的影响。

第六步：时间向前移动一小时

然后：

- 新水位变成当前状态；
- 上一步实际输入变成 `u_previous`；
- 预测窗口整体向前移动；
- 再求解一次 24 小时计划。

完整闭环就是：

测量 → 预测 → 优化 24 步 → 执行 1 步 → 更新水箱 → 重新优化

11. 现在再对应到代码结构

数学原理落在以下几个位置：

- 参数 $A, \Delta t, q_{\max}, N$ 、水位范围和权重，在 `basic_mpc_config.m`。
- 预测矩阵、目标函数、约束和 `quadprog`，在 `constrained_tank_mpc.m`。
- “规划 24 步但只执行一步”的滚动闭环，在 `simulate_basic_mpc.m`。
- 模拟需求和参考水位，在 `make_mock_decision_data.m`。
- 临时假设“实际流量完全等于要求流量”，在 `mock_decision_optimizer.m`。

12. 必须准确理解这版模型的边界

当前已经有：

- 单水箱动态模型；
- 未来 24 小时预测；
- 水位上下界；
- 流量上下界；
- 流量变化约束；
- 水位跟踪目标；
- 连续流量优化；
- 滚动时域闭环；
- MPC 与 Decision 之间的接口。

当前还没有：

- 电价成本；
- 电价预测；
- 水泵功率模型；
- 具体泵组开关；
- 二进制变量；
- 启停成本；
- 最小连续运行时间；
- 真实 Decision 执行误差；
- 不确定性或预测误差处理。

所以准确称呼应该是：

基础约束水箱 MPC，或者连续流量跟踪 MPC。

不能直接称为完整 EMPC。虽然数据结构里保留了 `price` 字段，但这个价格目前没有进入目标函数，对优化结果没有影响。

下一步读代码时，最合理的顺序是：

1. `basic_mpc_config.m`：先认清参数；

2. `make_mock_decision_data.m`: 理解控制器收到什么;
3. `simulate_basic_mpc.m`: 看完整闭环;
4. `constrained_tank_mpc.m`: 逐行拆解矩阵和 `quadprog`;
5. 最后看三个接口文件。